Performance Improvement

To improve database performance, we can focus on four levels of optimization:

1. Development Level ⇒ SQL Query
2. Procedural Level ⇒ Functions or Procedures
3. Architecture/DBA Level
4. Hardware Level

Development Level:

At the development level, focusing on optimizing SQL queries can greatly enhance database performance. Consider the following techniques:

Use Appropriate Indexing: Analyze the query execution plan and identify columns involved in frequently executed queries. Create indexes on these columns to speed up data retrieval.

Index Utilization:

- **Avoid unnecessary type conversions:** Minimize unnecessary type conversions in your queries whenever possible. Performing type conversions on indexed columns can potentially prevent the database optimizer from effectively using the index, impacting query performance.
- **Use compatible data types:** When comparing indexed columns, use compatible data types in your query conditions. Matching the data types allows the optimizer to utilize the index more efficiently. Casting the indexed column to a different data type may hinder index utilization.
- Indexes are typically created on columns that are frequently used in JOIN operations or columns involved in WHERE clauses to improve query performance
- The presence of unnecessary indexes can negatively impact the performance of insert, update, and delete operations, resulting in slower database operations overall

Minimize Data Retrieval:

- Retrieve only the necessary data by selecting specific columns instead of using "**SELECT** *". This reduces the amount of data transferred and improves query performance.

Optimize Joins: Use proper join techniques, such as inner joins, left joins, or right joins, depending on the data relationships. Ensure that join conditions are properly defined and optimized. Avoid unnecessary joins that can impact query performance.

- **Sequence:**
  - INNER JOIN ⇒ OUTER JOIN
  - INNER JOIN (Smaller Set) ⇒ INNER JOIN (Larger Set)
- **Advantage:** This approach can minimize the number of comparisons and improve performance. By reducing the number of records early in the query execution, subsequent joins can be performed on a smaller subset of data.

Analyzing Join Effectiveness: In addition to analyzing the indexes used, you should examine the effectiveness of join strategies decided by the optimizer. SQL Server uses four types of joins.

- Hash Joins:
  - Suitable for large tables or when joining tables with no or limited indexes.

    - Efficient for joining large datasets when memory is available for hash tables.
- Merge Joins:
    - Effective for joining two sorted datasets based on a common column.
    - Efficient when join columns are properly indexed and tables are sorted in the same order.
- Nested Loop Joins:
    - Suitable for small to medium-sized tables or selective criteria.
    - Efficient when join condition results in a small subset of data or when there's an index matching the join condition.
    - Nested loop joins are highly effective when the outer input table is small and the inner input table is larger but indexed
- Adaptive Joins:
    - Introduced in SQL Server 2017 to dynamically switch between join strategies.
    - Analyzes statistics and available resources to choose the most efficient join strategy (a nested loop join or a hash join) on the fly using adaptive threshold rows.
    - It's applicable only to columnstore indexes

JOIN Type	Index on Joining Columns	Usual Size of Joining Tables	Presorted JOIN Clause
Nested loops	Inner table a must	Small	Optional
	Outer table preferable		
Merge	Both tables a must	Large	Yes
	Optimal condition: clustered or covering index on both		
Hash	Inner table *not* indexed	Any	No
		Optimal condition: inner table large, outer table small	
Adaptive	Uses either hash or loops depending on the data being returned by the query	Variable, but usually very large because it currently works only with columnstore indexes	Depends on join type

Avoid Subqueries when Possible: Subqueries can be resource-intensive. Whenever feasible, rewrite subqueries as joins or use temporary tables to improve query performance.

- **Example:** Use Local (#) and Global (##) Temp Table
- Complex nested queries can be harder for the optimizer to optimize and can result in suboptimal query plans. Consider breaking down complex queries into multiple simpler queries or using common table expressions (CTEs) to improve query readability and performance.

Limit Data Manipulation Operations: Minimize the use of functions and processes within queries, as they can significantly impact performance. Instead, consider performing complex calculations or transformations at the application level.

- When complex functions or processes are involved, it becomes harder for the optimizer to accurately estimate the cost of different execution plans. This can result in suboptimal query plans and decreased performance
- By offloading these operations to the application layer, you can leverage the processing power of the application server or client devices, which are often more

flexible and scalable than database engines. Application servers and client devices can perform computations efficiently using programming languages and libraries optimized for such tasks.

- Application servers and client devices typically have more processing power and resources compared to database servers
- They often have multicore CPUs
- larger amounts of RAM, and dedicated GPUs for certain tasks
- Application servers and client devices can be easily scaled horizontally by adding more instances or distributed across multiple nodes or clusters. This scalability allows for distributed computing, where computations can be divided into smaller tasks and executed in parallel across multiple machines

Use Query Optimization Techniques: Familiarize yourself with the specific optimization techniques supported by your database system. These may include query hints, rewriting, or leveraging database-specific features to improve performance.

Analyze and monitor query performance: Regularly analyze query execution plans and monitor the performance of your queries. Identify slow-performing queries and optimize them based on the identified bottlenecks.

Properly Handle NULL Values: Be cautious when dealing with NULL values in queries. Ensure that appropriate conditions are used to handle NULLs efficiently, especially when working with JOINs or WHERE clauses.

- JOIN: Handle NULL if the joining column contains NULL values.
- WHERE: Handle NULL if where a column contains NULL values.
  - COALESCE etc

Denormalize: Denormalization can enhance query performance, especially for complex queries or scenarios where frequent joins across multiple tables are required. By reducing the number of joins and minimizing data retrieval operations, denormalization can speed up query execution

Exists and Not Exists:

- **Early Termination:** The EXISTS and NOT EXISTS operators use a technique called early termination or short-circuiting. They stop processing as soon as a matching or non-matching row is found, respectively. This means that once a match is found, the query execution can be terminated, reducing the overall processing time. In contrast, the IN and NOT IN operators may need to evaluate the entire list of values before returning the result.
- **Set Semantics:** EXISTS and NOT EXISTS are based on set semantics rather than value matching. They focus on the existence of a record in a subquery or a correlated subquery. In contrast, the IN and NOT IN operators compare individual values from a list. Set-based operations are generally more optimized and efficient for working with large datasets.
- **NULL Handling:** EXISTS and NOT EXISTS operators handle NULL values more intuitively than the IN and NOT IN operators. In some cases, the IN operator may produce unexpected results or require additional conditions to handle NULL values properly. EXISTS and NOT EXISTS provide a clearer and more consistent behavior when dealing with NULL values.

Use EXISTS over COUNT(*) to Verify Data Existence:

```
DECLARE @n INT;
SELECT @n = COUNT(*)
FROM Sales.SalesOrderDetail AS sod
WHERE sod.OrderQty = 1;
IF @n > 0
    PRINT 'Record Exists';
```

Using COUNT(*) to verify the existence of data is highly resource-intensive because COUNT(*) has to scan all the rows in a table. EXISTS merely has to scan and stop at the first record that matches the EXISTS criterion. To improve performance, use EXISTS instead of the COUNT(*) approach.

```
IF EXISTS (   SELECT sod.*
              FROM Sales.SalesOrderDetail AS sod
              WHERE sod.OrderQty = 1)
    PRINT 'Record Exists';
```

GROUP BY instead of DISTINCT:

- **Index Utilization:** When you use GROUP BY, the database engine has the opportunity to utilize indexes on the grouping columns. Indexes are data structures that allow for faster data retrieval by organizing the data in a specific order. By leveraging indexes, the database engine can quickly identify and group the unique values, resulting in improved performance.
- **Sorting Optimization:** GROUP BY implicitly sorts the data based on the grouping columns, which can be advantageous in some cases. If the data is already sorted in the required order or if the query execution plan incorporates an optimized sorting strategy, GROUP BY can take advantage of this and avoid an additional sorting step. On the other hand, DISTINCT typically requires a separate sorting operation to eliminate duplicate values.
- **Aggregation Benefits:** GROUP BY is often used in conjunction with aggregate functions, such as COUNT, SUM, AVG, etc., which can be more efficient than using DISTINCT. When you use DISTINCT, the database engine needs to identify and eliminate duplicate values for the entire result set. In contrast, when using GROUP BY with aggregate functions, the database engine performs the aggregation on the grouped data directly, potentially resulting in better performance.

Use Query Caching

- Query caching in SQL Server involves storing the execution plans of queries in memory for reuse.
- The execution plan for a query is generated by the query optimizer based on factors like table statistics, indexes, and configuration settings.
- The execution plan is stored in the plan cache, a portion of the SQL Server's memory.
- When a similar query is executed, SQL Server can reuse the cached execution plan instead of generating a new plan, leading to faster query execution.
- Query caching improves performance by reducing the CPU and memory overhead of query optimization and utilizing resources efficiently.
- The plan cache is dynamic and can be invalidated when data or schema changes occur to ensure accurate results.

- Query caching is specific to each SQL Server instance or database.
- Overall, query caching in SQL Server speeds up query execution and improves the performance of frequently executed queries.

Optimize Data Types:

- **Use Fixed-Length Data Types:** Improve performance ⇒ sorting and joining. However, be cautious when using fixed-length data types for variable-length data, as it can waste storage space.
- **Avoid Overly Large Data Types:** Avoid using unnecessarily large data types when the actual range of values is smaller.
- **Use Sparse Columns:** If you have columns with many NULL values, consider using sparse columns. Sparse columns optimize storage by only allocating space for non-NULL values, reducing the overall storage requirements and potentially improving query performance. However, sparse columns have certain limitations and should be used judiciously.
    - **Limitation:**
        - Sparse columns cannot be used in primary key or unique constraints.
        - Sparse columns cannot have certain data types, such as TEXT, IMAGE, TIMESTAMP, etc.
        - Sparse columns cannot be part of clustered indexes.
        - Sparse columns have additional overhead for maintaining the sparse structure, which may slightly impact update and insert performance.
        - **Supported DB:** MS SQL Server. PostgreSQL, Oracle automatically hande this type of NULL values by default.
- **Compress Data:** If your database system supports data compression, consider enabling compression for large tables or columns. Compression can reduce the storage footprint and improve I/O performance by reducing disk reads and writes. However, be mindful of the potential trade-offs in terms of increased CPU usage for compression and decompression operations.
    - **SQL Server:**
        - Row Compression and Page Compression.
        - These features can be applied to tables and indexes
        - Reducing storage requirements and potentially improving I/O performance
    - **Oracle:**
        - Table compression, index compression, and backup compression.
        - These compression techniques can help reduce storage space and improve performance
    - **PostgreSQL**:
        - Does not have built-in data compression features
        - It supports various compression techniques through extensions. The "pg_compression" and "pg_zstd" extensions, for example, provide compression capabilities for PostgreSQL tables.
- **Consider Data Type Conversion:** Evaluate the need for data type conversions in your queries. Unnecessary data type conversions can impact query performance. Minimize implicit conversions and explicitly convert data types only when necessary.
    - `CAST(Salary AS INT) > 50000`

Avoid Unnecessary Sorting:

- Minimize the use of sorting operations in your queries unless explicitly required.

- Sorting large result sets can be resource-intensive and impact query performance.
- If sorting is necessary, consider optimizing the query by using appropriate indexes or sorting techniques specific to your database system.
- Never use sorting in inner block of sub query. It's useless.

Limit the Use of Wildcards:

- Avoid using leading wildcards (e.g., %value) in your search patterns when using the LIKE operator. Leading wildcards prevent the efficient use of indexes and can result in slow query performance.
- Whenever possible, structure your queries to use trailing wildcards (e.g., value%) or full-text search features for better performance.
- Use IN () if it has exact values.

Use UNION ALL Instead of UNION if Table have No Duplicates.

Avoid Nonsargable Search Conditions: The sargable search conditions listed in Table generally allow the optimizer to use an index on the columns referred to in the WHERE clause. The sargable search conditions generally allow SQL Server to seek to a row in the index and retrieve the row (or the adjacent range of rows while the search condition remains true). On the other hand, the nonsargable search conditions listed in Table 19-1 generally prevent the optimizer from using an index on the columns referred to in the WHERE clause. The exclusion search conditions generally don't allow SQL Server to perform Index Seek operations as supported by the sargable search conditions. For example, the != condition requires scanning all the rows to identify the matching rows.

Table 19-1. *Common Sargable and Nonsargable Search Conditions*

Type	Search Conditions
Sargable	Inclusion conditions =, >, >=, <, <=, and BETWEEN, and some LIKE conditions such as LIKE '<literal>%'
Nonsargable	Exclusion conditions <>, !=, !>, !<, NOT EXISTS, NOT IN, and NOT LIKE IN, OR, and some LIKE conditions such as LIKE '%<literal>'

Procedural Level:

Execution Plans: Execution plans in SQL Server are valuable tools for identifying problematic queries. They help in query optimization by highlighting inefficient query constructs and suggesting improvements. Execution plans also identify performance bottlenecks by providing insights into resource consumption and query cost estimation. Additionally, they offer information on index usage and recommendations, aiding in optimizing query performance. In summary, execution plans are instrumental in diagnosing and resolving issues to enhance query efficiency in SQL Server.

Use Parameterized Procedures: Parameterized procedures accept input parameters, allowing you to pass values to the procedure during execution. Parameterization improves performance by reusing query plans and reducing the overhead of query compilation.

Avoid Excessive Use of Triggers: Triggers are automatically executed in response to certain database events. While triggers can be useful in certain scenarios, excessive use of

triggers can impact performance due to their overhead. Evaluate the necessity of triggers and use them judiciously.

Optimize Procedural Code: Review and optimize the code within your procedures. This includes avoiding unnecessary loops, reducing the number of temporary tables, optimizing the use of cursors, and minimizing unnecessary calculations or transformations.

Avoid Excessive Logging or Printing: Excessive logging or printing within procedures can impact performance due to I/O operations. Limit the use of logging or printing statements to essential information for debugging or auditing purposes.

Properly Manage Transactions: Ensure that transactions are appropriately managed within your procedures. Use transactions only when necessary and keep their scope as short as possible to reduce contention and improve concurrency.

Handle Errors Effectively: Implement error handling mechanisms within your procedures to handle exceptions and errors gracefully. Proper error handling helps prevent unexpected termination of procedures and improves overall reliability and performance.

Regularly Review and Optimize Procedures: Periodically review and optimize your procedures based on usage patterns and changing business requirements. Identify and eliminate any unnecessary or redundant code to improve performance.

Utilize Stored Procedure Caching: Many database systems cache the execution plans of stored procedures to improve performance. Take advantage of this caching mechanism by reusing stored procedures whenever possible, rather than executing ad-hoc SQL queries.

Used Preserve Table: Use preserve table which will be incrementally inserted instead of a full refresh.

Avoid Excessive Locking: Minimize the use of locking mechanisms within procedures to avoid contention and performance degradation. Consider using row-level locking instead of table-level locking where possible.

Use Table Variables instead of Temporary Tables: For smaller datasets, consider using table variables instead of temporary tables as they can have lower overhead and faster access.

```
DECLARE @Employees TABLE (EmployeeID INT, FirstName VARCHAR(50))
```

Avoid Excessive Looping: Whenever possible, avoid iterative processing within procedures. Instead, leverage set-based operations to process data in batches or in a single operation. A set-based operation refers to performing operations on an entire set of data at once, rather than processing data row by row. In other words, it involves treating a group of rows as a single unit and performing operations on them collectively.

- **Aggregation:** SUM, COUNT, AVG, MAX, and MIN to calculate aggregate values across a set of rows
- **Joins:** Combining data from multiple tables based on matching columns, such as INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL JOIN. Joins allow you to retrieve data from related tables in a single operation.
- **Filtering:** Applying WHERE or HAVING conditions to filter rows based on specific criteria. This allows you to extract subsets of data from a larger dataset.
- **Sorting:** Ordering the result set based on one or more columns using the ORDER BY clause. Sorting can be done in ascending or descending order.
- **Updating:** Updating multiple rows in a table based on specified conditions using the UPDATE statement. This allows you to modify data across a set of rows simultaneously.
- **Deleting:** Removing multiple rows from a table based on specified conditions using the DELETE statement. This allows you to delete data in bulk rather than individually.

Optimize Cursor Usage: If you need to use cursors within your procedures, ensure

they are properly optimized, using appropriate fetch sizes and cursor options to minimize overhead.

Analyze and Optimize Procedure Dependencies: Review the dependencies between your procedures and identify any potential bottlenecks or areas for optimization. Ensure efficient data flows and minimize unnecessary procedure calls.

Regularly Monitor and Tune Procedure Performance: Continuously monitor the performance of your procedures using database performance monitoring tools. Identify slow-performing procedures and optimize them based on identified bottlenecks.

Regularly Update Statistics: Keep database statistics up to date to ensure the query optimizer makes accurate decisions about query plans and index usage within your procedures.

Architecture/DBA Level:

Here are the details of performance enhancement at the Architecture/DBA Level in SQL Server:

Database Design: Ensure a well-designed database schema that follows normalization principles and minimizes redundancy. This includes properly defining tables, columns, relationships, and constraints to optimize data storage and retrieval.

Indexing Strategy: Create appropriate indexes on columns frequently used in queries to improve data retrieval performance. Identify columns used in WHERE, JOIN, and ORDER BY clauses, and create indexes to speed up these operations.

Index Maintenance: Regularly monitor and maintain indexes by rebuilding or reorganizing them to optimize their efficiency. This involves analyzing index fragmentation levels and performing maintenance tasks accordingly.

Partitioning: Implement table partitioning to manage large tables and improve query performance. Partitioning involves splitting a table into smaller, more manageable segments based on a defined partitioning key, such as a date or a range of values.

Database Statistics: Keep database statistics up to date to help the query optimizer generate efficient execution plans. Outdated or inaccurate statistics can lead to suboptimal query plans and performance degradation. Regularly update statistics using the UPDATE STATISTICS command or automate the process.

Query Optimization: Analyze and fine-tune query execution plans using tools like SQL Server Query Store or Execution Plans. Identify queries with high resource consumption or slow performance, and optimize them by adding appropriate indexes, rewriting the query logic, or using query hints.

Stored Procedure Optimization: Optimize frequently executed stored procedures by reviewing their execution plans and making necessary adjustments. This can involve optimizing query logic, parameterization, or recompiling stored procedures to ensure optimal performance.

Query Tuning: Identify and optimize slow-performing queries using techniques like adding appropriate indexes, rewriting the query logic, or leveraging query optimization features like query hints, query plan guides, or plan forcing.

Inaccurate Statistics:

- Accurate data distribution statistics are crucial for SQL Server's query optimizer to estimate the number of rows affected by a query effectively.
- Outdated or inaccurate statistics hinder the optimizer's ability to make informed decisions, leading to suboptimal query plans and reduced performance.
- Skewed data distribution can distort statistics, resulting in inaccurate cardinality estimates and inefficient query plans.

- Columns with auto-incrementing values, like dates, may have outdated statistics if not regularly updated, affecting query optimization.
- Regularly updating statistics is essential to ensure optimal query performance and efficient utilization of indexes in SQL Server.

Database Configuration: Review and optimize SQL Server configuration settings based on workload and hardware resources. Adjust settings such as memory allocation, parallelism, maximum degree of parallelism (MAXDOP), and other configuration options to align with the specific requirements of your workload.

Memory Configuration: Configure the maximum memory limit for SQL Server to ensure efficient memory utilization. Set an appropriate maximum server memory setting to prevent SQL Server from consuming all available memory and causing resource contention with other applications.

Tempdb Optimization: Properly configure and optimize the tempdb database for improved performance. This includes setting appropriate initial size, autogrowth settings, and placing tempdb on separate disks for better I/O performance.

Database Maintenance: Schedule regular database maintenance tasks like backup, integrity checks, and index defragmentation. Regular maintenance ensures data consistency, prevents corruption, and optimizes database performance.

Query Tracing: Use tools like SQL Server Profiler or Extended Events to capture and analyze query performance. Tracing allows you to identify and analyze queries with high resource consumption, long duration, or frequent execution.

Deadlock Analysis: Monitor and analyze deadlock occurrences to identify and resolve concurrency issues. Use SQL Server Profiler, Extended Events, or system_health Extended Events session to capture deadlock information and analyze the deadlock graphs.

Blocking Analysis: Identify and resolve blocking issues by monitoring and analyzing blocking chains. SQL Server's built-in dynamic management views (DMVs) and Extended Events can provide information about current blocking chains and help identify the causes.

Query Plan Caching: Enable query plan caching to reuse execution plans and avoid unnecessary query compilation overhead. SQL Server's plan cache stores execution plans for frequently executed queries, allowing for plan reuse and improved performance.

Data Compression: Utilize data compression techniques to reduce storage requirements and improve I/O performance. SQL Server offers row compression and page compression options.

Log File Management: Properly manage transaction log files by sizing them appropriately and regularly backing them up to maintain performance.

Resource Governor: Resource Governor: Utilize SQL Server's Resource Governor feature to allocate and manage server resources effectively. Resource Governor allows you to prioritize resource allocation for different workloads, ensuring critical queries receive adequate resources and preventing resource contention.

Database Mirroring or Always on Availability Groups: Implement high availability solutions to ensure database availability and minimize downtime. These solutions provide failover capabilities that can help maintain performance during server failures or planned maintenance.

Database Replication: Utilize replication to distribute data and workload across multiple servers for improved performance and scalability.

SQL Server Agent Jobs: Optimize and schedule SQL Server Agent jobs to minimize their impact on server performance and ensure they run at appropriate times to avoid conflicts with other critical tasks and maintenance activities. Additionally, regularly review and fine-tune the jobs' execution plans, data retrieval methods, and resource utilization to enhance overall database performance and efficiency.

SQL Server Version Upgrade: Consider upgrading to the latest version of SQL Server, which often includes performance improvement impact on server performance during peak hours.

Database Scaling: Consider scaling up (vertical scaling) or scaling out (horizontal scaling) your database infrastructure to handle increasing workloads. Vertical scaling involves upgrading hardware resources, such as CPU, memory, or storage, to boost performance. Horizontal scaling involves distributing the workload across multiple servers to improve scalability.

Database Sharding: Implement database sharding techniques to horizontally partition data across multiple databases or servers. Sharding allows for distributed data storage and query processing, enabling better performance and scalability for large-scale systems.

Database Archiving: Implement a database archiving strategy to move infrequently accessed or historical data to separate storage tiers. Archiving reduces the size of the active database, improving query performance on the most frequently accessed data.

Database Monitoring and Alerting: Set up proactive monitoring and alerting mechanisms to detect and respond to performance issues promptly. Utilize tools like SQL Server's built-in monitoring features, performance counters, and external monitoring solutions to receive alerts on critical performance metrics.

Query Parallelism: Query parallelism in SQL Server optimizes database query performance by allowing simultaneous execution, improving resource utilization, and enabling scalability. It divides the workload among multiple processors, reducing execution time, maximizing CPU usage, and dynamically adjusting to handle increased demands efficiently.

Hardware Level:

Typically, SQL Server database performance is affected by stress on the following hardware resources:

- *Memory*
- *Disk I/O*
- *Processor*
- *Network*

Once you have identified bottlenecks, you can resolve them in two ways:

- You can increase resource capacity.
- You can decrease the arrival rate of requests to the resource

High CPU: Need more CPU. More clock speed. CPU will read data from disk. So disk must be faster as weel. Then processed data from disk will be faster.

Upgrade Hardware Components: Upgrade the hardware components of your database server, such as CPU, memory (RAM), and storage (SSD or NVMe drives), to increase processing power, improve memory capacity, and enhance I/O performance.

Memory Performance: Experiencing I/O contention: (these listed in the top 10)

- **ASYNC_IO_COMPLETION**: This wait event indicates that a task is waiting for an asynchronous I/O operation to complete. It typically occurs when data is being read from or written to disk asynchronously.
- **IO_COMPLETION:** This wait event signifies that a task is waiting for an I/O operation to complete. It can occur when reading or writing data from disk, network communication, or other I/O-related activities.
- **LOGMGR:** The LOGMGR wait event indicates that a task is waiting for the log manager to complete a transaction log operation. It typically occurs during the writing or flushing of transaction log records to disk.
- **WRITELOG:** This wait event occurs when a task is waiting for the log writer to flush log buffers to disk. It indicates that the system is waiting for the completion of write operations to the transaction log.

- **PAGEIOLATCH:** This wait event signifies that a task is waiting for a latch on a data page to be released. It occurs when a task needs to read or write data from/to disk and is waiting for the necessary data page to be loaded into memory or written back to disk.

Results | Messages

	wait_type	waiting_tasks_count	wait_time_ms	max_wait_time_ms	signal_wait_time_m
1	SOS_WORK_DISPATCHER	196398544	9403803963	50926024	394564471
2	SLEEP_TASK	413909	823952848	50918654	43629494
3	LOGMGR_QUEUE	1654634	606052465	50917904	7586
4	CLR_AUTO_EVENT	4654	605985271	50972400	722
5	DISPATCHER_QUEUE_SEMAPHORE	1985	306694991	50925777	1448
6	HADR_FILESTREAM_IOMGR_IOCOMPLETION	227696	303032347	50918195	43622947
7	XE_TIMER_EVENT	7683	303031874	50947647	302919365
8	QDS_PERSIST_TASK_MAIN_LOOP_SLEEP	1943	303025219	50946802	69
9	REQUEST_FOR_DEADLOCK_SEARCH	23197	303024429	50922654	303024429
10	QDS_ASYNC_QUEUE	5	303023754	224896900	0

High-Speed Storage:

- NVMe (Non-Volatile Memory Express) SSD
- PCIe (Peripheral Component Interconnect Express) SSD
- SATA (Serial ATA) SSD
- iSCSI interfaces offer faster disk I/O operations
- Traditional platter-style disk drives like HDD are becoming obsolete.

Disk Bottleneck Resolutions

- Using a faster I/O path: Ensuring that your I/O subsystem, including disks and storage controllers, is capable of handling the workload efficiently. This may involve using high-performance disk drives, optimizing RAID configurations, or utilizing faster storage technologies such as NVMe SSDs.
- Using a RAID array: Implementing a redundant array of independent disks (RAID) can improve both performance and fault tolerance. RAID configurations distribute data across multiple disks, allowing for parallel I/O operations and enhancing data protection.
- Using a SAN system: A storage area network (SAN) provides centralized storage resources accessed by multiple servers. SANs offer high-speed and shared storage, enabling improved performance and scalability for application workloads.
- Using solid-state drives (SSDs): Replacing traditional hard disk drives (HDDs) with solid-state drives can significantly enhance I/O performance. SSDs offer faster read and write speeds, lower latency, and better random access times, resulting in improved application responsiveness.
- Aligning disks properly: Ensuring that partitions and file systems are properly aligned on disk blocks can optimize I/O performance. Alignment ensures that data is accessed more efficiently and minimizes unnecessary disk operations.
- Adding system memory: Increasing the amount of memory available to the system can reduce the need for disk I/O operations. More memory allows for better caching of frequently accessed data, reducing disk access latency.
- Creating multiple files and filegroups: Distributing database files across multiple disks or filegroups can improve I/O parallelism. This strategy allows for simultaneous read and write operations on different disks, enhancing overall performance.

- Moving the log files to a separate physical drive: Separating transaction log files onto a dedicated disk can improve write performance and reduce contention with data files. This ensures that log writes are not impeded by other disk operations.
- Using partitioned tables: Partitioning large tables based on specific criteria, such as date or range, can enhance query performance. Partition elimination allows the database to scan only relevant partitions, reducing I/O and improving overall query execution time.

RAID Configuration: Implement RAID (Redundant Array of Independent Disks) configurations for improved data storage and fault tolerance

- *RAID 0*: Striping with no fault tolerance
- *RAID 1:* Mirroring
- *RAID 5:* Striping with parity
- *RAID 1+0*: Striping with mirroring

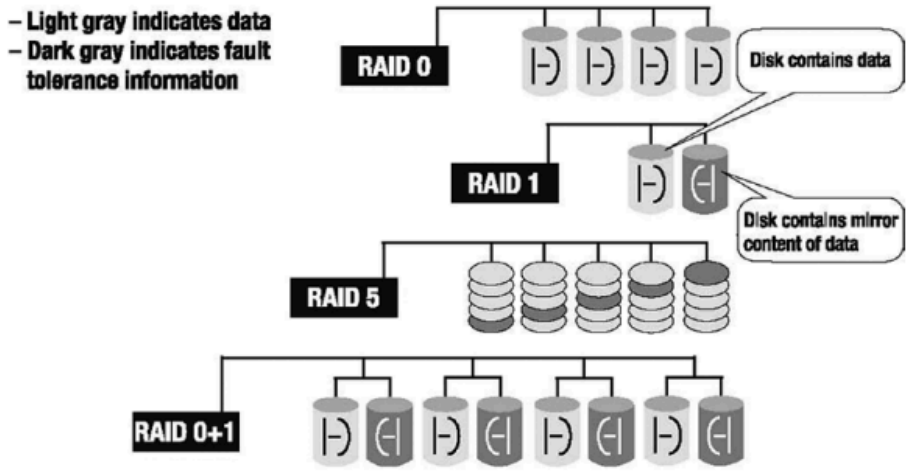


Figure 3-1. *RAID configurations*

- **RAID 0:** I/Os per disk = (Reads + Writes) / Number of disks in the array
  ⇒ RAID 0 array with 4 disks and you perform 200 reads and 400 writes
  ⇒ I/Os per disk = (200 + 400) / 4
  = 600 / 4
  = 150
- **RAID 1:** I/Os per disk =(Reads + 2 * Writes) / 2
  ⇒ Suppose you have a RAID 1 array with 2 disks, and you perform 100 reads and 200 writes:
  ⇒ I/Os per disk = (Reads + 2 * Writes) / 2
  = (100 + 2 * 200) / 2
  = (100 + 400) / 2
  = 500 / 2
  = 250
- **RAID 5:** I/Os per disk = (Reads + 4 * Writes) / Number of disks in the array
  ⇒ Suppose you have a RAID 5 array with 5 disks, and you perform 200 reads and 300 writes:
  ⇒ I/Os per disk = (Reads + 4 * Writes) / Number of disks in the array

= (200 + 4 * 300) / 5
= (200 + 1200) / 5
= 1400 / 5
= 280

- **RAID 6:** I/Os per disk = (Reads + 6 X Writes) / (Number of disks in the array 2)
  ⇒ Suppose you have a RAID 6 array with 8 disks, and you perform 300 reads and 500 writes:
  ⇒ I/Os per disk = (Reads + 6 * Writes) / (Number of disks in the array 2)
  = (300 + 6 * 500) / (8 2)
  = (300 + 3000) / 6
  = 3300 / 6
  = 550
- **RAID 1+0 (RAID 10):** I/Os per disk = (Reads + 2 X Writes) / Number of disks in the array
  ⇒ Suppose you have a RAID 10 array with 4 disks, and you perform 150 reads and 300 writes:
  ⇒ I/Os per disk = (Reads + 2 * Writes) / Number of disks in the array
  = (150 + 2*300) / 4
  = (150 + 600) / 4
  = 750 / 4
  = 187.5

Memory Configuration: Optimize memory configuration settings, such as the maximum server memory and memory allocation for SQL Server. Adjusting these settings ensures that SQL Server has sufficient memory to cache frequently accessed data and queries, reducing disk I/O and improving performance.

CPU Affinity: Set CPU affinity to allocate specific CPU cores or processors to SQL Server. This ensures that SQL Server has dedicated CPU resources, minimizing CPU contention and improving overall performance.

Temperature and Cooling: Ensure proper server cooling and temperature management to prevent overheating. Elevated temperatures can lead to performance degradation or even hardware failures. Maintain a well-ventilated server room and monitor server temperatures regularly.

High-speed Interconnects: Utilize high-speed interconnect technologies, such as InfiniBand or 10GbE/40GbE networking, for communication between database servers and storage systems. These technologies provide low latency and high bandwidth, reducing data transfer bottlenecks.

RAID Table:

RAID Level	Description	Fault Tolerance	Read Performance	Write Performance	Disk Space Utilization
RAID 0	Striped disk array without redundancy	No	High	High	100%
RAID 1	Mirrored disk array	Yes (1 disk failure)	High (Reads)	Low (Writes)	50%
RAID 5	Striped disk array with distributed parity	Yes (1 disk failure)	High (Reads)	Medium (Writes)	(N-1)/N (where N is the number of disks)
RAID 6	Striped disk array with dual distributed parity	Yes (2 disk failures)	High (Reads)	Medium (Writes)	(N-2)/N (where N is the number of disks)
RAID 10	Mirrored and striped disk array	Yes (Multiple disk failures in different mirrors)	High (Reads)	High (Writes)	50% (half the capacity of total disks)

Finding Costly Queries

Extended Events:

- Identify the costly procedure from Extended Events using the `rpc_completed` event or other relevant events capturing query execution information.
- Once you have identified the procedure, you can further analyze its performance by collecting additional data points such as duration, CPU time, logical reads, and physical reads.
- Use Extended Events to capture additional events or data related to the procedure execution, such as blocking or waits, to gather more insights into the performance issues.
- Analyze the captured data to identify potential performance bottlenecks, such as excessive reads, long duration, high CPU usage, or blocking, that may be causing the query to be costly.

Query Store:

- Query the Query Store to identify the costly query or procedure. The Query Store retains historical information about query performance, execution plans, and resource usage.
- Retrieve the execution statistics, including duration, CPU time, logical reads, and physical reads, for the identified query or procedure from the Query Store.
- Analyze the execution statistics to understand the performance characteristics of the query, such as high resource usage or long duration, and compare it with the desired performance baseline.
- Examine the associated execution plans stored in the Query Store to identify any plan changes or suboptimal plans that may be contributing to the query's poor performance.
- Consider using the Query Store's built-in reports or querying the Query Store tables directly to gain more insights into the query's performance over time.

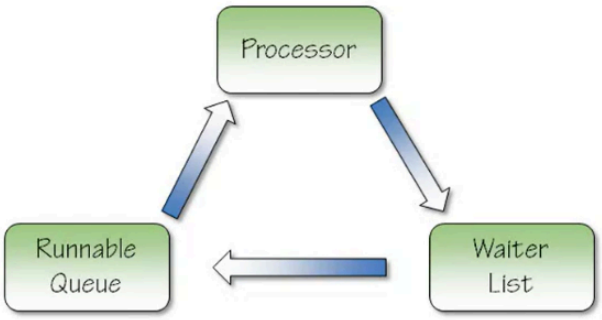
sys.dm_exec_procedure_stats:

- Query the `sys.dm_exec_procedure_stats` dynamic management view to retrieve execution statistics for stored procedures.
- Identify the procedure with high resource usage or long duration based on the available statistics, such as total_elapsed_time, total_worker_time, logical_reads, or physical_reads.
- Analyze the execution statistics to understand the performance characteristics of the procedure and compare them to the desired performance baseline.
- Consider examining the associated execution plans for the procedure using other dynamic management views, such as `sys.dm_exec_query_stats`, to identify any plan issues or optimizations that can improve its performance.

Different Wait Type Analysis

Introduction

All schedulers are composed of three parts



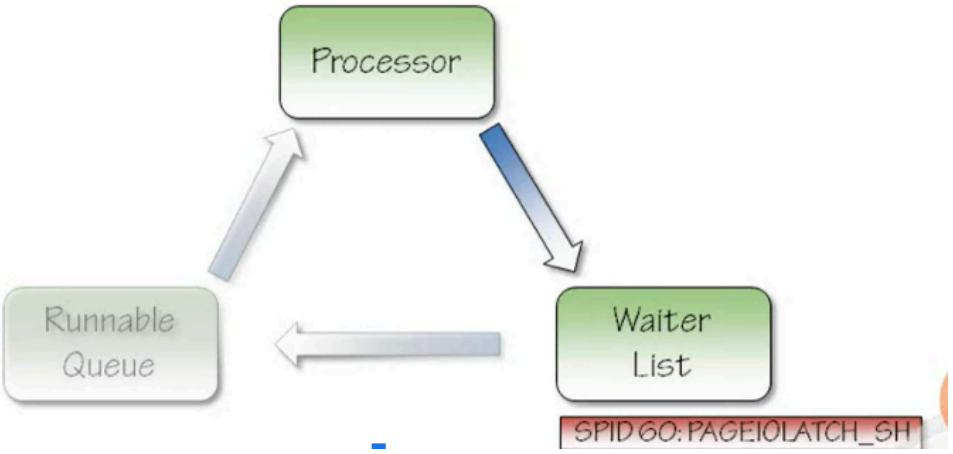
Threads transition around these parts until their work is complete.

A thread can be in one of three states when being actively used as part of processing a query

- **RUNNING:** The thread is currently executing on the processor
- **SUSPENDED:** The thread is currently on the Waiter List waiting for a resource
- **RUNNABLE:** The thread is currently on the Runnable Queue waiting to execute on the processor

A thread continues executing on the processor until it must wait for a resource to become available

- The thread's state changes from RUNNING to SUSPENDED
- The thread moves to the Waiter List
- This process is called being ‘suspended’



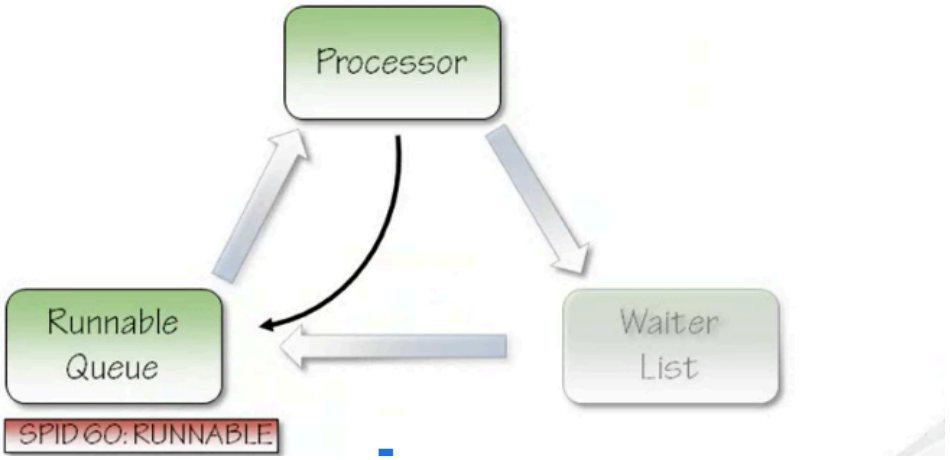
The Waiter List

- The Waiter List is an unordered list of threads that are suspended
- Any thread can be notified at any time that the resource it is waiting for is now available

    - Again, absolutely no ordering
- There is no limit to how long a thread remains on the waiter list
    - Although execution timeouts or lock timeouts may take effect
- There is no practical limit to how many threads can be on a scheduler's waiter list at any time
- The sys.dm_os_waiting_tasks DMV shows which threads are currently waiting and what they are waiting for

Quantum Exhaustion:

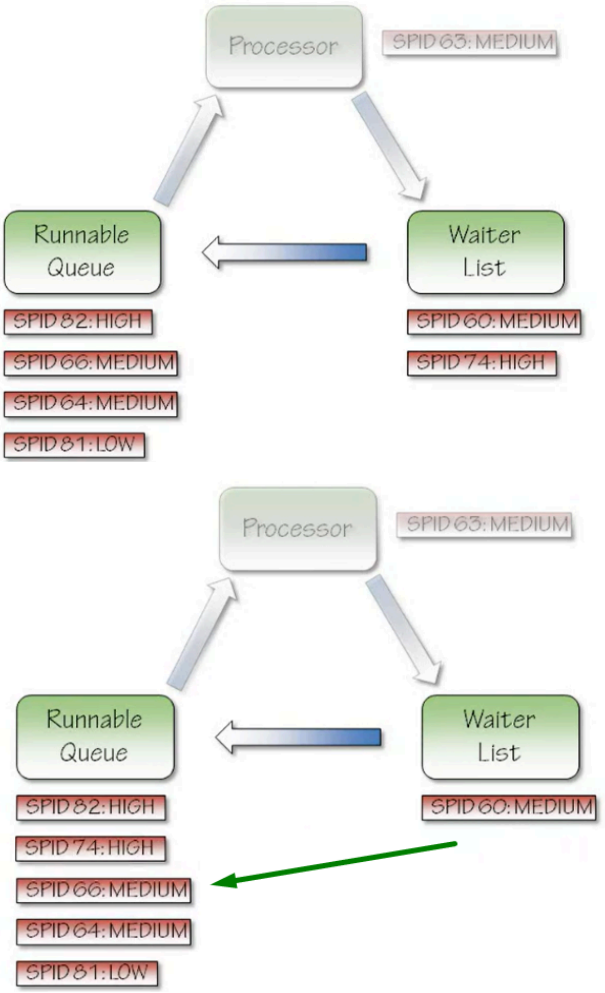
- This type of thread does not need to wait. So it will directy moved in Runnable Queue.
- Thread quantum is fixed at 4 milli seconds and cant be changed.
- Runnable Queue maintains FIFO.



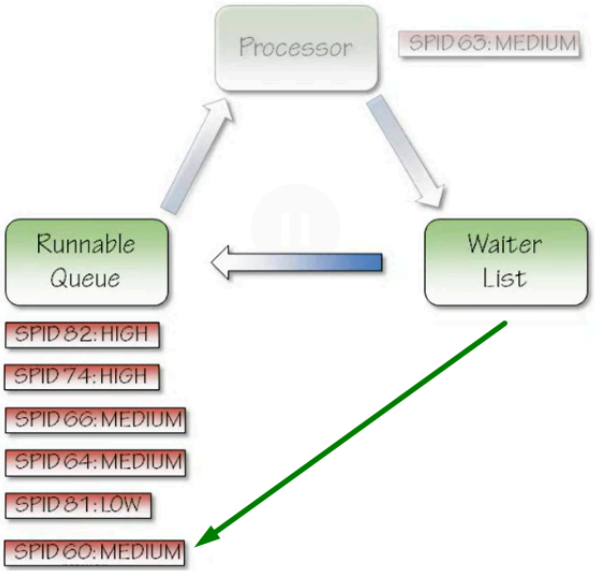
Resource Governor:

- Using Resource Governor (in Enterprise Edition of SQL Server 2008 onwards) it is possible to define multiple workload groups that share the same resource pool
    - A workload group can be thought of as a bucket of connections
    - A resource pool can be thought of as a set of CPU and memory limits, but more details are beyond the scope of this course
- Multiple workload groups for a resource pool can be assigned relative priorities
    - The default priority is medium
    - Possible values are high, medium, and low
- The relative priorities change how the Runnable Queue works
    - High to medium to low equates to **9 to 3 to 1** in terms of the priority of the threads that are permitted to execute
    - For example, for each low-priority thread on the runnable queue, nine high-priority threads will be allowed to jump over it and execute first

Case 1: High ⇒ 1+0 (Runnable + Processor) < 9 (True). So High from Waiter list will move to last of High's



Case 2: Mediam⇒ 2+1 (Runnable + Processor) < 3 (False). So Mediam from Waiter list will move to last of the queue.



Wait Statistics

Three Types of Latches

1. PAGEIOLATCH_XX
2. PAGELATCH_XX
3. LATCH_XX

Top Wait Types:

1. CXPACKET
2. OLEDB
3. WRITELOG
4. ASYNC_NETWORK_IO
5. PAGEIOLATCH_XX
6. BACKUPXX
7. SOS_SCHEDULER_YIELD
8. PAGELATCH_XX
9. LCK_M_XX

CXPACKET

What does it mean:

- Parallel operations are taking place
- Accumulating very fast implies skewed work distribution amongst threads or
- one of the workers is being blocked by something

Avoid knee-jerk response:

- Do not set server-wide MAXDOP to 1, disabling parallelism

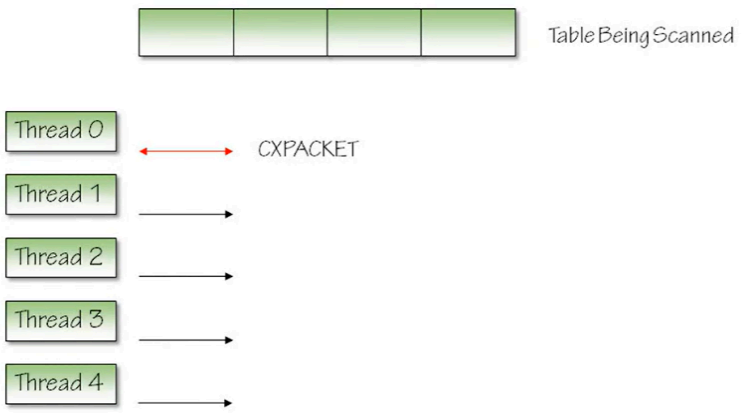
Further analysis:

- Correlation with PAGEIOLATCH_SH waits? Implies large scans
- Examine CXPACKET query plan to see if the query plans make sense
- What is the wait type of the non-control parallel thread that is taking too long?
- (i.e. the thread that does not have CXPACKET as its wait type)

Example

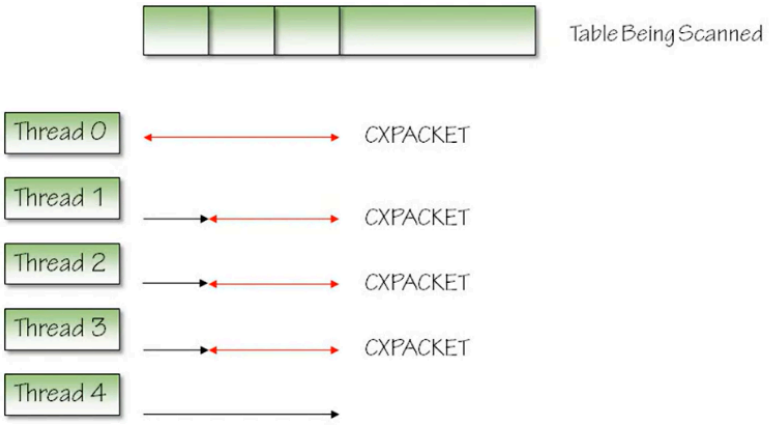
Case 1:

- 1 Co-Ordinate threads and 4 workers threads.
- In that case only co ordinate threads is the root cause.



Case 2:

- 1 Co-Ordinate threads and 4 workers threads.
- In that case co ordinate threads along with 3 worker threads are causes. .



Possible root-causes:

- Just parallelism occurring (e.g. http://bit.ly/1kPymkZ from CSS)
- Table scans due to missing nonclustered indexes or incorrect query plan
- Out-of-date statistics causing skewed work distribution

If there is actually a problem:

- Make sure statistics are up-to-date and appropriate indexes exist
- Consider MAXDOP = physical cores per NUMA node, or 8 for non-NUMA
- Consider MAXDOP for the instance, but beware of mixed workloads
- Consider MAXDOP for the query or using Resource Governor for MAX_DOP
- Consider setting ‘cost threshold for parallelism’ higher than query cost

OLEDB

What does it mean:

- Object Linking and Embedding Database (OLEDB) is a mechanism used within SQL Server to connect to other database resources
- The OLE DB mechanism is being used

Avoid knee-jerk response:

- Do not assume that linked servers are being used

Further analysis:

- What are the queries doing that are waiting for OLEDB?
- If Linked servers are being used, what is causing the delay on the linked server?

Possible root-causes:

- DBCCCHECKDB and related commands use OLE DB internally
- Many DMVs use OLE DB internally so it could be a third-party monitoring tool that is repeatedly calling DMVs
- Poor performance of a linked server

WRITELOG

What does it mean:

- Waiting for a transaction log block buffer to flush to disk

Avoid knee-jerk response:

- Do not assume that the transaction log file I/O system has a problem (although this is often the case)
- Do not create additional transaction log files

Further analysis:

- Correlate WRITELOG wait time with I/O subsystem latency using *sys.dm_io_virtual_file_stats* a Look for LOGBUFFER waits, showing internal contention for log buffers
- Look at average disk write queue length for log drive
- If constantly 31/32 then the internal limit has been reached for outstanding transaction log writes for a single database
- Look ataveragesize of transactions
- Investigate whether frequent page splits are occurring

Solution:

- Move the log to a faster I/O subsystem
- Increase the size of transactions to prevent many minimum-size log block flushes to disk
- Remove unused nonclustered indexes to reduce logging overhead from maintaining unused indexes during DML operations
- Change index keys or introduce FILLFACTORs to reduce page splits
- Potentially split the workload over multiple databases or servers

ASYNC_NETWORK_IO

What does it mean:

- SQL Server is waiting for a client to acknowledge receipt of sent data

Avoid knee-jerk response:

- Do not assume that the problem is network latency
- Its a network delay as far as SQL Server is concerned though

Further analysis:

- Analyze client application code
- Analyze network latencies

Possible root-causes and solutions:

- Nearly always a poorly-coded application that is processing results one record ata time (RBAR = Row-By-Agonizing-Row)
    - Very easy to demonstrate using a large query and SQL Server Management Studio running on the same machine as SQL Server
- Otherwise look for network hardware issues, incorrect duplex settings, or TCP chimney offload problems (see http://bit.ly/aPzoAx)

PAGEIOLATCH_XX

What does it mean:

- Waiting for a data file page to be read from disk into memory
- Common modes to see are SH and EX
    - SH mode meansthe page will be read
    - EX mode means the page will be changed

Avoid knee-jerk response:

- Do not assume the I/O subsystem or I/O path is the problem

Further analysis:

- Determine which tables/indexes are being read
- Analyze I/O subsystem latencies with *sys.dm_io_virtual_file_stats* and Avg Disk secs/Read performance counters
- Correlate with CXPACKET waits, suggesting parallel scans
- Examine query plans for parallel scans
- Examine query plans for implicit conversions
- Investigate buffer pool memory pressure and Page Life Expectancy.

Solution:

- Create appropriate nonclustered indexes to reduce scans
- Update statistics to allow efficient query plans
- Move the affected data files to faster I/O subsystem
- If data volume has simply increased, consider increasing memory

BACKUPXX

Examples:

- BACKUPBUFFER
- BACKUPIO
- BACKUPTHREAD

What do they mean:

- BACKUPBUFFER ⇒ The first is **waiting for data** or a **buffer for data**
- BACKUPIO ⇒ The second is **reading from database data files**
- BACKUPTHREAD ⇒ The third is usually while **waiting for some I/O to complete** on another thread, like zero initialization of a data or log file

Further analysis and root causes:

- BACKUPBUFFER ⇒ For the first, are the **backups going to a slow tape** or **over a slow network**? Is the I/O subsystem on the remote server slow?
- BACKUPIO ⇒ For the second, is **the I/O subsystem for the data files slow**?
- BACKUPTHREAD ⇒ For the third, see the **PREEMPTIVE_OS_WRITEFILEGATHER** wait

SOS_SCHEDULER_YIELD

What does it mean:

- A thread exhausted its 4 millisecond quantum and voluntarily yielded
- A threadis spinning on a spinlock and backing off every so often

Avoid knee-jerk response:

- Do not assume that CPU pressure is the problem

Further analysis:

- Examine query plans to see whether scans are occurring
- Look to see whether there is a very small or non-existent number of PAGEIOLATCH_XX waits occuring, which indicates that the workload is memory: resident a Look for long Runnable Queues
- Capture SQL Server code call stacks to see where the waits are occurring

Note: these waits have zero resource wait time so regular methods of aggregating and prioritizing waits will miss them

- They Don't appear in sys.dm_os_waiting_tasks

Possible root-causes:

- SQL Server is executing code that can use a lot of CPU without having to wait for a resource (e.g. a large scan with few PAGEIOLATCH_SH waits)
- Look Also for long Runnable Queues, indicating CPU pressure
- Spinlock contention

Solutions:

- For the quantum exhaustion case on slower processors, potentially enable hyper-threading to give more schedulers and more potential for concurrent work, especially for OLTP workloads
- For The spinlock case, the solution varies by spinlock a Seethe common spinlocks slide near the end of the module for more information
- More often than not, call stack analysis shows that spinlocks are notinvolved.

PAGELATCH_XX

What does it mean:

- Waiting for access to an in-memory data file page
- Common modes to see are SH and EX
- SH mode meansthe page will be read
- EX mode means the page will be changed

Avoid knee-jerk response:

- Do not confuse these with PAGEIOLATCH_XX waits
- Does not mean add more memory or I/O capacity

Further analysis:

- Determine the page(s) that the thread is waiting for access to
- Analyze the queries encountering this wait
- Analyze the table and index structures involved

Solution:

Classic tempdb contention

- Add more tempdb datafiles
- Enable trace flag 1118
- Reduce temp table usage

Excessive page splits occurring in indexes

- Change to a non-random index key
- Avoid updating index records to be longer
- Provision an index FILLFACTOR to alleviate page splits

Insertion point hotspot in a clustered index with an ever-increasing key

- Spread the insertion points in the index using a random or composite key, plus provision a FILLFACTOR to prevent page splits
- Shard into multiple tables/databases/servers

LCK_M_XX

What does it mean:

- A thread is waiting for a lock that cannot be granted because another thread is holding an incompatible lock

Avoid knee-jerk response:

- Don't assume that locking is the root cause

Further analysis:

- Follow the blocking chain using ***sys.dm_os_waiting_tasks*** to see what the lead blocking thread is waiting for
- Use the blocked process report to capture information on queries waiting too long for locks

Solution:

Lock escalation from a large update or table scan

- Possibly configure partition-level lock escalation, if applicable
- Consider a differentindexing strategy to use nonclustered index seeks
- Consider breaking large updates into smaller transactions
- Consider using snapshot isolation, a different isolation level, or locking hints
- All the general strategies for alleviating blocking problems

Unnecessary locks for the data being accessed

- Consider using snapshotisolation
    - Readers dont block Writers
    - Writers dont block readers
- Different isolation level (Read Uncommited)
- Locking hints

Something preventing a transaction from releasing its locks quickly

- Determine what the bottleneck is and solve it appropriately
- A transaction releases its lock promptly but experiences delays in committing due to lock wait caused by synchronous database mirroring, potentially resulting in blocking issues. These delays may arise from network issues during the mirroring process.

Deep Understanding

- ➢ **SQL Server Architectue**
- ➢ **Hands-On Exercise**
- ➢ **Performance Analysis**